

Análise de Resiliência e Propagação de Vulnerabilidades em Grafos de Dependências do Ecossistema Python

Márcio Levi Sales Prado — RA 183680

MC859 — Projeto em Teoria da Computação

Professor: Ruben Interian

Repositório do github: https://github.com/marcinholsp/Projeto_MC859

1. Resumo

Este trabalho investiga o risco de supply chain no Python Package Index (PyPI), onde a comprometimento de um pacote amplamente reutilizado pode afetar milhares de projetos dependentes. O ecossistema é modelado como um grafo direcionado de dependências, cruzando-se a topologia da rede, dados reais de downloads e vulnerabilidades catalogadas pelas bases OSV e GitHub Advisory; a propagação de falhas é simulada por um modelo de Independent Cascade. O grafo capturado reúne 54.406 nós e 178.193 arestas, revelando uma rede scale-free essencialmente acíclica, com 648 pacotes vulneráveis. Os pacotes mais críticos identificados são urllib3, numpy e pyyaml. A detecção de comunidades aponta 63 agrupamentos com modularidade de 0,50, e a validação do modelo de propagação atinge Recall@50 igual a 1,0, confirmando a aderência das previsões aos pacotes efetivamente mais expostos.

2. Introdução

O ecossistema PyPI caracteriza-se por um grau elevado de interdependência: a grande maioria dos projetos reaproveita funcionalidades de bibliotecas de terceiros, que por sua vez dependem de outras, formando longas cadeias de reuso. Nesse contexto, uma vulnerabilidade introduzida em um pacote-hub não permanece isolada; ela se propaga, direta ou transitivamente, para todos os projetos que dele dependem, configurando o que se convencionou chamar de risco de supply chain. A onipresença de pacotes como urllib3 e requests, presentes na base de instalação de incontáveis aplicações, ilustra como uma única falha pode adquirir alcance praticamente universal dentro do ecossistema.

Para quantificar esse fenômeno, o problema é tratado como um grafo direcionado em que cada vértice representa um pacote e cada aresta $A \rightarrow B$ significa que A depende de B. Essa formulação permite aplicar métricas consolidadas da teoria dos grafos — medidas de centralidade, análise de componentes e detecção de comunidades — em conjunto com um modelo de cascata, de modo a mensurar tanto a resiliência estrutural da rede quanto a extensão potencial da propagação de uma falha a partir de qualquer ponto de entrada.

O objetivo geral é identificar padrões de propagação de vulnerabilidades e pacotes de alto risco no PyPI por meio da teoria dos grafos, desdobrado em cinco objetivos específicos: (i) construir o grafo de dependências do ecossistema; (ii) identificar os pacotes de maior centralidade; (iii) mapear as vulnerabilidades conhecidas a partir das bases OSV e GitHub Advisory; (iv) simular falhas e medir o efeito cascata resultante; e (v) detectar e visualizar as comunidades de pacotes mais afetadas.

O restante do documento organiza-se da seguinte forma. A Seção 3 descreve a captura de dados e a construção do grafo; a Seção 4 apresenta os algoritmos e a metodologia adotados; a Seção 5 analisa a topologia da rede; a Seção 6 examina as vulnerabilidades e a propagação de falhas; a Seção 7 trata da detecção de comunidades; a Seção 8 discute a validação do modelo; e a Seção 9 reúne as conclusões do trabalho.

3. Captura de dados

A construção do grafo de dependências partiu de uma estratégia de amostragem orientada por impacto, e não de uma varredura exaustiva do PyPI. O repositório contém, na prática, centenas de milhares de pacotes, a maioria dos quais com baixíssimo uso e influência marginal sobre o ecossistema. Adotou-se, portanto, como conjunto prioritário de sementes a lista dos 8.000 pacotes mais baixados (top-pypi-packages, distribuída em hugovk.github.io, com janela de contagem de 30 dias), complementada por 40.000 pacotes adicionais amostrados do PyPI Simple Index, respeitando um teto de 100.000 nós. A justificativa é dupla: os pacotes mais baixados concentram os hubs de alto

grau de entrada — bibliotecas das quais inúmeros outros projetos dependem direta ou indiretamente — de modo que captá-los garante a presença dos vértices estruturalmente mais relevantes para qualquer análise de propagação; já a amostra do Simple Index amplia a cobertura da periferia e reduz o viés de selecionar apenas pacotes populares, sem incorrer no custo proibitivo de processar o índice inteiro.

A coleta dos metadados de cada pacote foi realizada por meio da API JSON do PyPI (<https://pypi.org/pypi/<pkg>/json>). De cada resposta extraiu-se o campo `requires_dist`, do qual se consideraram exclusivamente as dependências de runtime, ignorando-se deliberadamente as dependências opcionais declaradas como extras. Essa decisão é metodologicamente importante: extras descrevem funcionalidades acessórias que só são instaladas mediante solicitação explícita do usuário (por exemplo, `requests[security]`) e, portanto, não refletem o que de fato é instalado por padrão quando um pacote é adicionado a um ambiente. Incluí-las inflaria o grafo com arestas que raramente se materializam em instalações reais, distorcendo as medidas de alcance e de propagação. Restringir o grafo às dependências de runtime produz, assim, uma representação mais fiel do que efetivamente se propaga pelo ecossistema em condições típicas de uso.

Os nomes de pacotes foram normalizados conforme a PEP 503 — conversão para minúsculas e colapso de hífens, sublinhados e pontos em um separador canônico —, etapa indispensável para a deduplicação correta dos vértices. Sem ela, denominações equivalentes como “PyYAML” e “pyyaml”, ou “typing_extensions” e “typing-extensions”, seriam tratadas como nós distintos, fragmentando o grafo e subestimando o grau dos hubs. A expansão do ecossistema deu-se por uma busca em largura (BFS) transitiva a partir das sementes: a cada pacote visitado, suas dependências de runtime eram enfileiradas e, se ainda não vistas, agendadas para coleta. A execução empregou 20 threads paralelas (ThreadPoolExecutor) com deduplicação dos nós já processados, um `REQUEST_DELAY` de 0,05 s entre chamadas, timeout de 10 s e até 3 tentativas por pacote, conferindo robustez frente a falhas transitórias da rede e respeitando os limites de uso da API. O procedimento completo de construção corresponde ao Algoritmo 1, apresentado na seção de Algoritmos.

Concluída a expansão, cada vértice foi anotado com dois conjuntos de atributos. O primeiro registra a popularidade, medida pelo número de downloads mensais segundo a base `top-pypi-packages`; pacotes fora dessa lista receberam o valor zero. O segundo agrega os metadados de vulnerabilidade — a contagem de CVEs associadas e o CVSS máximo observado —, obtidos da base `OSV/GitHub Advisory` por meio da biblioteca `cvss`. Essa anotação dupla é o que viabiliza, nas seções seguintes, cruzar a topologia do grafo com a relevância e a exposição de segurança de cada pacote.

A Tabela 1 resume as características finais do grafo resultante. Os artefatos foram serializados nos formatos `pypi_dependency_graph.graphml` (aproximadamente 23 MB) e `pypi_dependency_graph.gexf` (aproximadamente 31 MB), preservando rótulos e atributos dos nós para uso nas etapas analíticas subsequentes.

Tabela 1 — Métricas estruturais do grafo de dependências capturado.

Métrica	Valor
Vértices (pacotes)	54.406
Arestas (dependências de runtime)	178.193
Grau médio	6,55 (in = out = 3,28)
In-degree máximo	6.479 (numpy)
Out-degree máximo	1.000
Componentes fortemente conexas (Tarjan)	54.298 (54.238 singletons; maior CFC = 11)
Componentes fracamente conexas	14.003 (maior = 40.105 vértices; 73,7% do grafo)
Nós com ≥ 1 vulnerabilidade conhecida	648 (1,2% do total)
Artefatos gerados	GraphML (~23 MB) e GEXF (~31 MB)

Cumprе ressaltar que o grafo capturado constitui um subconjunto do PyPI, e não o ecossistema em sua totalidade. Em consequência, qualquer métrica de alcance, de propagação ou de tamanho de componente derivada deste grafo deve ser interpretada como um limite inferior em relação ao ecossistema completo: a inclusão de pacotes adicionais só poderia preservar ou ampliar os caminhos de dependência observados, jamais reduzi-los.

4. Algoritmos (Metodologia)

Esta seção descreve os seis algoritmos centrais do projeto, organizados segundo as três etapas do pipeline: a construção e a análise estrutural do grafo de dependências (Algoritmos 1 e 2); a modelagem de propagação de falhas e a quantificação de risco (Algoritmos 3 e 4); e a detecção de estrutura comunitária e a validação do modelo

(Algoritmos 5 e 6). Para cada um apresentam-se a motivação e a escolha de projeto, o pseudocódigo correspondente e uma nota de complexidade. Mantém-se a notação adotada ao longo do trabalho: $G=(V,E)$ é o grafo direcionado de dependências, em que a aresta (a,b) indica que a depende de b ; G_{rev} é o grafo reverso; $dl(v)$ e $cvss(v)$ denotam, respectivamente, os downloads mensais e a severidade CVSS máxima de v ; e $\log_1p(x) = \ln(1 + x)$.

Algoritmo 1 — Construção do grafo de dependências por BFS transitivo paralelo

A construção do grafo parte de uma busca em largura (BFS) transitiva a partir de sementes de alto impacto — os pacotes mais baixados unidos a uma amostra do Simple Index —, reconstruindo o fecho transitivo de dependências sem varrer todo o PyPI. A paralelização em até vinte threads amortiza a latência da API JSON, e as dependências declaradas como extras são ignoradas por não refletirem instalações padrão.

Require: lista de sementes S (top- N por downloads u Simple Index); limites $MaxNodes$, $MaxWorkers$

Ensure: grafo direcionado $G=(V,E)$ com aresta (a,b) sse a depende de b

```

1: procedure BuildGraph(S)
2:    $G \leftarrow$  grafo direcionado vazio
3:    $seen \leftarrow \emptyset$ ;  $processed \leftarrow \emptyset$ 
4:    $batch \leftarrow \emptyset$  ▷ fronteira do nível atual: pares (pkg, is_seed)
5:   for each  $p \in S$  do
6:      $q \leftarrow \text{Normalize}(p)$  ▷ normalização PEP 503
7:     if  $q \notin seen$  then
8:        $seen \leftarrow seen \cup \{q\}$ ; adicionar  $(q, \text{true})$  a batch
9:   while  $batch \neq \emptyset$  do
10:     $toFetch \leftarrow \{ (p,s) \in batch : p \notin processed \}$ 
11:     $processed \leftarrow processed \cup \{ p : (p,s) \in toFetch \}$ 
12:    if  $toFetch = \emptyset$  then break
13:     $next \leftarrow \emptyset$ 
14:    em paralelo, com até  $MaxWorkers$  threads, para cada  $(p,s) \in toFetch$ :
15:       $info[p] \leftarrow \text{FetchPackageInfo}(p)$  ▷ apenas HTTP nas threads
16:      for each  $(p,s) \in toFetch$  na ordem de conclusão do ▷ atualização do grafo na thread principal
17:        if  $info[p] = \text{nil}$  then continue
18:         $a \leftarrow info[p].name$ 
19:        adicionar nó  $a$  a  $G$  com atributos {version, summary, home_page, is_seed=s}
20:        for each  $b \in info[p].deps$  do
21:          adicionar aresta  $(a,b)$  a  $G$ 
22:          if  $b \notin seen$  and  $|seen| < MaxNodes$  then
23:             $seen \leftarrow seen \cup \{b\}$ ; adicionar  $(b, \text{false})$  a next
24:         $batch \leftarrow next$  ▷ avança um nível na largura
25:   return  $G$ 

26: function FetchPackageInfo(p) ▷ API JSON do PyPI, com até MaxRetries tentativas
27:   consultar pypi.org/pypi/p/json; se 404 então return nil
28:    $r \leftarrow$  campo info.requires_dist (ou  $\emptyset$ )
29:   return {name, version, summary, home_page, deps = ParseRequiresDist(r)}

30: function ParseRequiresDist(R) ▷ dependências de runtime (PEP 508)
31:    $D \leftarrow \emptyset$ 
32:   for each especificação  $spec \in R$  do
33:     if  $spec$  contém "extra ==" then continue ▷ ignora deps opcionais de extras
34:      $n \leftarrow$  primeiro token de  $spec$  antes de [\s([;<=&;!~@]
35:     if  $n \neq \text{vazio}$  then adicionar  $\text{Normalize}(n)$  a  $D$ 
36:   return  $D$ 

```

Complexidade. Tempo $O(|V| + |E|)$ em chamadas/processamento (uma requisição HTTP por nó descoberto, dominada pela latência de rede e mitigada pelas $MaxWorkers$ threads); espaço $O(|V| + |E|)$ para o grafo e o conjunto $seen$.

Algoritmo 2 — Tarjan: Componentes Fortemente Conexas (CFCs)

Para caracterizar a topologia do grafo, aplica-se o algoritmo de Tarjan, que detecta componentes fortemente conexas — isto é, ciclos de dependência — em tempo linear $O(|V|+|E|)$. O resultado obtido (99,89% de componentes singleton) é justamente o que autoriza afirmar que o grafo de dependências do PyPI é, na prática, essencialmente acíclico.

Require: grafo direcionado $G=(V,E)$

Ensure: partição de V em componentes fortemente conexas (CFCs)

```

1:  $index \leftarrow 0$ ;  $stack \leftarrow$  pilha vazia;  $CFCs \leftarrow \emptyset$ 
2:  $idx[v] \leftarrow \text{indef}$ ,  $low[v] \leftarrow \text{indef}$ ,  $onStack[v] \leftarrow \text{false}$  para todo  $v \in V$ 
3: for each  $v \in V$  do
4:   if  $idx[v] = \text{indef}$  then StrongConnect(v)
5: return CFCs

6: procedure StrongConnect(v)
7:    $idx[v] \leftarrow index$ ;  $low[v] \leftarrow index$ ;  $index \leftarrow index + 1$ 
8:   empilhar  $v$  em  $stack$ ;  $onStack[v] \leftarrow \text{true}$ 
9:   for each  $w \in succ(v)$  do
10:    if  $idx[w] = \text{indef}$  then
11:      StrongConnect(w);  $low[v] \leftarrow \min(low[v], low[w])$ 

```

```

12:     else if onStack[w] then
13:         low[v] ← min(low[v], idx[w])
14:     if low[v] = idx[v] then           ▷ v é raiz de uma CFC
15:         C ← ∅
16:         repeat
17:             w ← desempilhar stack; onStack[w] ← false; C ← C ∪ {w}
18:         until w = v
19:         CFCs ← CFCs ∪ {C}

```

Complexidade. Tempo $O(|V| + |E|)$ (cada vértice e aresta visitados uma vez); espaço $O(|V|)$ para os vetores $idx/low/onStack$ e a pilha. As componentes fracamente conexas (CFCs) são obtidas analogamente por busca no grafo subjacente não-direcionado, também em $O(|V|+|E|)$.

Algoritmo 3 — Independent Cascade ponderado por downloads (Monte Carlo)

A propagação de uma falha é modelada por um processo probabilístico de Independent Cascade, e não por um BFS determinístico: o alcance puramente estrutural superestima o impacto real, ao supor que toda dependência transmite a falha com certeza. Ponderar a probabilidade de propagação $p(B \rightarrow A)$ pela popularidade (downloads) aproxima a chance de que uma falha efetivamente atinja um dependente relevante; o alcance esperado é estimado por Monte Carlo, com 500 simulações por pacote-fonte.

```

Require: grafo  $G=(V,E)$  com  $dl(\cdot)$ ; pacote-fonte  $v_0$ ; nº de simulações  $NSims$ 
Ensure: alcance médio reach e downloads afetados médios  $dl$ 
1: function ComputeICProbabilities(G)           ▷ pré-computa p no grafo reverso
2:   for each  $a \in V$  do
3:      $W[a] \leftarrow \max(1, \sum_{b \in succ(a)} \log_{1p}(dl(b)) )$   ▷ evita divisão por zero
4:   for each aresta  $(a,b) \in E$  do
5:      $p[(b,a)] \leftarrow \min(1, \log_{1p}(dl(b)) / W[a] )$   ▷ fração do peso de dependência de a vinda de b
6:   return p

7: function ICSimulate( $v_0, G_{rev}, p, dl, NSims$ )
8:   reachList ← ∅; dlList ← ∅
9:   for  $t \leftarrow 1$  to  $NSims$  do
10:    infected ← { $v_0$ }; Q ← fila com  $v_0$ ; totalDl ← 0
11:    while Q ≠ ∅ do
12:      u ← desenfileirar Q
13:      for each  $w \in vizinhos_{G_{rev}}(u)$  do           ▷ w depende de u
14:        if  $w \notin infected$  and  $rand() < p[(u,w)]$  then
15:          infected ← infected ∪ {w}; totalDl ← totalDl + dl(w); enfileirar w
16:    anexar |infected| - 1 a reachList; anexar totalDl a dlList
17:   return ( Mean(reachList), Std(reachList), Mean(dlList) )

```

Complexidade. Pré-computação $O(|V| + |E|)$. Cada simulação percorre, no pior caso, $O(|V| + |E|)$; total $O(NSims \cdot (|V| + |E|))$ por pacote-fonte ($NSims=500$). Espaço $O(|V|+|E|)$ (probabilidades de aresta mais a fila). Um BFS determinístico adicional (não mostrado) fornece o alcance estrutural máximo para comparação.

Algoritmo 4 — Cálculo do escore de risco (risk_score) dos pacotes vulneráveis

O escore de risco combina três fatores que, isoladamente, seriam insuficientes: a gravidade técnica da vulnerabilidade (CVSS), o impacto prático esperado (downloads afetados, estimados pela cascata) e a relevância do próprio pacote (seus downloads). A composição multiplicativa via produto de $\log_{1p}$ garante que apenas pacotes simultaneamente graves, influentes e amplamente usados alcancem os primeiros lugares do ranking.

```

Require: grafo  $G=(V,E)$  com  $cvss(\cdot)$  e  $dl(\cdot)$ ;  $NSims$ 
Ensure: lista de pacotes vulneráveis ordenada por risk decrescente
1:  $G_{rev} \leftarrow$  reverso de  $G$ ;  $p \leftarrow$  ComputeICProbabilities( $G$ )
2:  $Vuln \leftarrow \{ v \in V : cvss(v) > 0 \}$ 
3: rows ← ∅
4: for each  $v \in Vuln$  do
5:   ( $reach, \sigma_{reach}, dl$ ) ← ICSimulate( $v, G_{rev}, p, dl, NSims$ )
6:    $risk(v) \leftarrow cvss(v) \cdot \log_{1p}(dl) \cdot \log_{1p}(dl(v))$ 
7:   anexar ( $v, cvss(v), dl(v), reach, dl, risk(v)$ ) a rows
8: ordenar rows por risk em ordem decrescente
9: return rows

```

Complexidade. Dominada pelas simulações IC: $O(|Vuln| \cdot NSims \cdot (|V| + |E|))$ tempo e $O(|V| + |E|)$ espaço.

Algoritmo 5 — Louvain: detecção de comunidades por maximização de modularidade

Para revelar a organização funcional do ecossistema, emprega-se o método de Louvain, que identifica módulos maximizando a modularidade. O procedimento roda sobre o maior componente fracamente conexo tratado como não-direcionado, pois a noção de comunidade — coesão interna superior à esperada ao acaso — é intrinsecamente não-orientada.

```

Require: grafo direcionado  $G$ ; semente Seed
Ensure: partição  $\pi$  (nó  $\mapsto$  id de comunidade, inteiro  $\geq 0$ ) e modularidade global  $Q$ 

```

```

1:  $H \leftarrow \text{ToUndirectedLargestWCC}(G)$   $\triangleright$  maior comp. fracamente conexo, não-direcionado
2: atribuir cada nó de  $H$  à sua própria comunidade
3: repeat
   $\triangleright$  Fase 1: otimização local gananciosa
4:   melhorou  $\leftarrow$  false
5:   for each nó  $i \in H$  (em ordem) do
6:     remover  $i$  de sua comunidade
7:     for each comunidade  $C$  vizinha de  $i$  do
8:       calcular  $\Delta Q$  de inserir  $i$  em  $C$ 
9:        $C^* \leftarrow \text{argmax}_C \Delta Q(C)$ 
10:      if  $\Delta Q(C^*) > 0$  then
11:        mover  $i$  para  $C^*$ ; melhorou  $\leftarrow$  true
   $\triangleright$  Fase 2: agregação (coarsening)
12:  construir novo grafo: cada comunidade vira um super-nó; pesos de aresta = soma dos pesos
  entre/dentro das comunidades
13:   $H \leftarrow$  grafo agregado
14:  until melhorou = false  $\triangleright Q$  não aumenta mais
15:   $\pi \leftarrow$  partição desdobrada de volta aos nós originais  $\triangleright$  nós fora do maior WCC recebem -1
16:   $Q \leftarrow \text{Modularity}(\pi, H)$   $\triangleright H =$  grafo não-direcionado original
17: return  $(\pi, Q)$ 

```

Complexidade. Empírica $\sim O(|V| \log |V|)$ tempo; espaço $O(|V| + |E|)$.

Algoritmo 6 — Precision@K e Recall@K do ranking por risk_score

Por fim, o ranking de risco é validado contra um gabarito de pacotes com incidentes reais por meio das métricas Precision@K e Recall@K, que medem quantos dos pacotes de fato problemáticos aparecem entre os K primeiros colocados. Essa avaliação é complementada pela correlação de Spearman entre o risk_score e proxies independentes (downloads e CVSS), descrita adiante.

```

Require: ranking rows (decrecente por risk_score); gabarito GT; cortes  $K = \{10, 20, 50\}$ 
Ensure: para cada K: Precision@K, Recall@K e nº de acertos
1: gtSet  $\leftarrow \{ \text{lower}(p) : p \in GT \}$ 
2: ranked  $\leftarrow [ \text{lower}(r.\text{package}) : r \in \text{rows} ]$ 
3: for each  $K \in K$  do
4:   topK  $\leftarrow$  primeiros K elementos de ranked
5:   hits  $\leftarrow |\{ p \in \text{topK} : p \in \text{gtSet} \}|$ 
6:   Precision@K  $\leftarrow \text{hits} / K$ 
7:   Recall@K  $\leftarrow \text{hits} / |\text{gtSet}|$ 
8: return  $\{ (K, \text{Precision@K}, \text{Recall@K}) \}$  para  $K \in K$ 

```

Complexidade. $O(|\text{rows}| + |GT|)$ tempo (construção dos conjuntos) mais $O(K_{\max})$ por corte; espaço $O(|\text{rows}|)$.

5. Análise topológica

A caracterização topológica do grafo de dependências constitui o primeiro passo para compreender a dinâmica de risco do ecossistema PyPI. O grafo capturado reúne 54.406 vértices e 178.193 arestas direcionadas, o que corresponde a um grau médio de 6,55 — repartido igualmente entre grau de entrada e de saída ($\text{in} = \text{out} = 3,28$), como decorrência natural do fato de que cada aresta contribui uma unidade para o grau de saída de sua origem e uma unidade para o grau de entrada de seu destino. Trata-se, portanto, de uma rede bastante esparsa: a razão entre o número de arestas observado e o número máximo possível em um grafo direcionado dessa ordem é ínfima, comportamento típico de ecossistemas de software, nos quais cada pacote declara apenas um punhado de dependências diretas, independentemente do tamanho total da rede.

Embora o grau médio seja baixo, a distribuição de graus está longe de ser homogênea. A Figura 1 apresenta a distribuição de graus em escala logarítmica nos dois eixos (log-log), tanto para o grau de entrada (in-degree, isto é, o número de pacotes que dependem de um dado pacote) quanto para o grau total. Em ambos os painéis a nuvem de pontos alinha-se aproximadamente a uma reta descendente, assinatura característica de uma distribuição em lei de potência. As inclinações ajustadas por regressão linear no espaço log-log são de aproximadamente $-0,91$ para o in-degree e $-1,20$ para o grau total. Esse comportamento qualifica o grafo como uma rede livre de escala (scale-free): a probabilidade de um pacote possuir grau k decai polinomialmente com k , de modo que poucos vértices — os hubs — concentram a imensa maioria das conexões, enquanto a vasta maioria dos pacotes apresenta grau muito baixo.

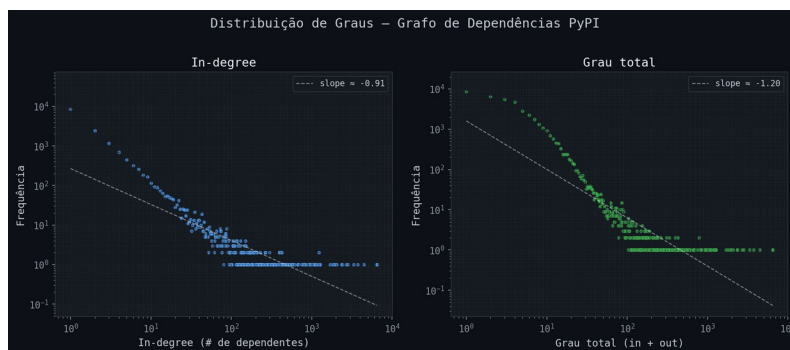


Figura 1. Distribuição de graus do grafo de dependências em escala log-log. À esquerda, o grau de entrada (in-degree); à direita, o grau total. As retas tracejadas indicam o ajuste em lei de potência, com inclinações de aproximadamente $-0,91$ e $-1,20$, respectivamente.

A topologia livre de escala tem implicações diretas sobre a resiliência da rede. Redes desse tipo são notoriamente robustas a falhas aleatórias — a remoção de um vértice escolhido ao acaso quase sempre atinge um pacote de grau baixo, sem efeito apreciável sobre a conectividade global — mas, em contrapartida, são fragilíssimas a falhas dirigidas: o comprometimento deliberado de um hub pode desarticular ou contaminar uma fração desproporcional do ecossistema. Essa assimetria entre robustez e vulnerabilidade é precisamente o que torna a identificação dos hubs uma questão de segurança, e não apenas de descrição estatística.

A Figura 2 elenca os pacotes mais dependidos do grafo, ordenados por grau de entrada. O vértice de maior in-degree é numpy, do qual dependem 6.479 pacotes, seguido de perto por requests (6.463). Completam a lista pydantic (4.461), pandas (3.746), pyyaml (3.348), typing-extensions (2.829), httpx (2.692), click (2.375), scipy (2.345) e matplotlib (2.060). Esses dez pacotes formam o núcleo estrutural do ecossistema: são bibliotecas de computação numérica, requisições HTTP, validação de dados e utilidades de tipagem que sustentam, direta ou indiretamente, milhares de outros projetos. Não por acaso, são exatamente os pontos cuja falha — seja por um defeito, seja por um comprometimento malicioso — teria o maior alcance potencial, conectando diretamente esta caracterização topológica à análise de vulnerabilidades desenvolvida na seção seguinte.

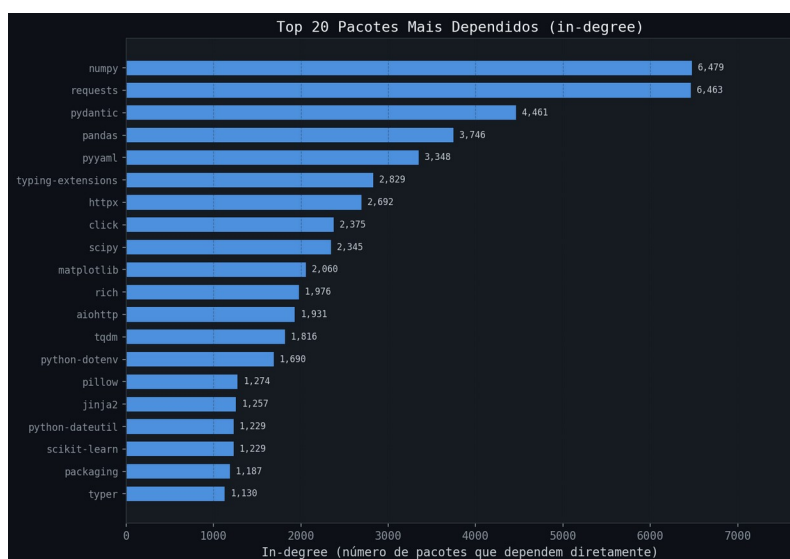


Figura 2. Pacotes mais dependidos do grafo, ordenados pelo grau de entrada (in-degree). numpy e requests lideram com mais de 6.400 dependentes cada, evidenciando a concentração de conexões em poucos hubs.

A análise das componentes fortemente conexas (CFCs), obtidas pelo algoritmo de Tarjan, revela uma segunda propriedade estrutural relevante. O grafo decompõe-se em 54.298 CFCs, das quais 54.238 — ou 99,89% do total — são componentes triviais, contendo um único vértice (singletons). A maior componente fortemente conexa reúne apenas 11 vértices. Como uma CFC com mais de um vértice exige a existência de um ciclo direcionado, a esmagadora predominância de singletons indica que o grafo é essencialmente acíclico: dependências circulares são raríssimas no ecossistema. Esse achado é coerente com as boas práticas de empacotamento em Python, que desencorajam ciclos de dependência por dificultarem a resolução de versões e a instalação, e permite tratar a rede, para fins práticos, como um grafo direcionado quase acíclico.

Diferente é o quadro quando se examinam as componentes fracamente conexas (CFrCs), isto é, as componentes obtidas ignorando-se a orientação das arestas. O grafo possui 14.003 componentes fracamente conexas, mas a distribuição de seus tamanhos, exibida na Figura 3, é extremamente desbalanceada: uma única componente gigante concentra 40.105 vértices, ou 73,7% de todo o grafo, enquanto as demais componentes são pequenas e periféricas. Em outras palavras, a maior parte do ecossistema forma um único bloco interconectado, no qual praticamente todo pacote alcança ou é alcançado por uma vasta fração dos demais quando se desconsidera o sentido das dependências. A consequência para a segurança é imediata: uma falha originada em um hub central pode, em princípio, propagar-se a dezenas de milhares de pacotes dentro dessa componente gigante — motivação direta para o modelo de propagação de falhas apresentado na Seção 6.

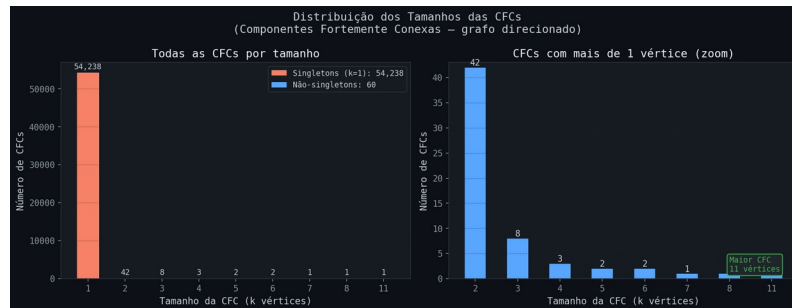


Figura 3. Distribuição do tamanho das componentes conexas do grafo. A esmagadora maioria das componentes fortemente conexas é trivial (singletons), ao passo que uma única componente fracamente conexa gigante abrange cerca de 73,7% dos vértices.

Em conjunto, a topologia livre de escala — com hubs altamente conectados — e a existência de uma componente fracamente conexa gigante delineiam um ecossistema cuja eficiência e interdependência são também sua principal fragilidade. A concentração de dependências em poucos pacotes críticos, somada à conectividade global do grafo, configura o cenário exato em que um incidente de cadeia de suprimentos (supply chain) pode escalar de um único pacote comprometido a um efeito sistêmico. É esse risco que as seções seguintes passam a quantificar, modelando a propagação de falhas e estimando o alcance esperado a partir dos pacotes mais centrais e vulneráveis.

6. Análise de vulnerabilidades e propagação

As seções anteriores caracterizaram a topologia do grafo de dependências e identificaram os pacotes estruturalmente centrais. Esta seção fecha o ciclo do projeto: parte do mapa de centralidade e o cruza com informação de segurança real, perguntando não apenas quais pacotes são importantes, mas quais deles combinam importância, fragilidade conhecida e capacidade de arrastar consigo grande parte do ecossistema caso sejam comprometidos. É nesse cruzamento que reside a noção de risco de cadeia de suprimentos (supply chain) que motivou todo o trabalho.

O levantamento de vulnerabilidades foi construído a partir das bases públicas OSV e GitHub Advisory, que consolidam avisos de segurança associados a versões de pacotes do PyPI. Dos 54.406 pacotes que compõem o grafo, apenas 648 — cerca de 1,2 % do total — possuem ao menos uma vulnerabilidade conhecida registrada nessas bases. Esse número, à primeira vista modesto, é enganoso: como se verá, vários desses 648 pacotes ocupam posições de altíssima centralidade, de modo que sua fração no conjunto de vértices subestima largamente sua influência sobre o ecossistema. Para cada pacote vulnerável adotou-se, como medida de gravidade, o maior valor de CVSS entre todas as suas vulnerabilidades, extraído de forma padronizada com a biblioteca cvss. O CVSS máximo é uma escolha conservadora e deliberada: do ponto de vista de um atacante, basta a falha mais severa para que o pacote represente um vetor de comprometimento, e portanto é essa que deve governar a avaliação de risco.

Conhecer a gravidade de cada pacote isoladamente, porém, não responde à pergunta central do projeto, que é de natureza sistêmica: se um pacote vulnerável for comprometido, quão longe a falha pode se propagar? Uma dependência comprometida não afeta apenas quem a importa diretamente, mas potencialmente toda a árvore de pacotes que dela dependem de forma transitiva. Para quantificar esse efeito de contágio, adotou-se um modelo de Cascata Independente (Independent Cascade, IC) ponderado pela popularidade dos pacotes, executado sobre o grafo de dependências invertido — isto é, no sentido em que uma falha "sobe" de um pacote para os que dependem dele.

No modelo, a probabilidade de uma falha presente em um pacote B se propagar até um dependente A é definida por $p(B \rightarrow A) = \log_{10}(\text{downloads}_B) / \sum_{D \in \text{deps}(A)} \log_{10}(\text{downloads}_D)$, limitada a 1. A intuição é direta: entre todas as dependências de A, aquelas mais amplamente adotadas tendem a ocupar papel mais crítico no seu funcionamento, e portanto uma falha nelas tem maior chance de efetivamente impactar A. Como não há, em escala de ecossistema, medida direta da criticidade de cada dependência, a popularidade — aproximada pelo número de downloads —

funciona como proxy razoável desse peso relativo. A transformação logarítmica ($\log_{1p}$) comprime a enorme variação de downloads, que abrange várias ordens de grandeza, evitando que um único pacote hegemônico domine completamente a soma, ao mesmo tempo em que preserva a ordenação por popularidade. A normalização pela soma sobre todas as dependências de A garante que $p(B \rightarrow A)$ expresse o peso de B relativo ao conjunto de dependências de A, e não um valor absoluto arbitrário. O alcance esperado de cada pacote-fonte foi então estimado por simulação de Monte Carlo, com 500 propagações independentes por fonte; a média do número de pacotes atingidos ao longo dessas simulações constitui o alcance IC reportado.

Gravidade técnica e alcance esperado são, por fim, combinados em um único escore que ordena os pacotes por prioridade de mitigação. O escore de risco de um pacote v é dado por $\text{risk_score}(v) = \text{CVSS}(v) \cdot \log_{1p}(\text{downloads afetados pelo IC}) \cdot \log_{1p}(\text{downloads próprios de } v)$. Os três fatores capturam dimensões complementares e necessárias do risco. O CVSS expressa a gravidade técnica da falha; o termo $\log_{1p}(\text{downloads afetados pelo IC})$ traduz o impacto prático esperado da propagação, ponderando o alcance não pelo simples número de pacotes atingidos, mas pelo volume de downloads que eles representam — ou seja, pelo tamanho real da população exposta; e o termo $\log_{1p}(\text{downloads próprios de } v)$ incorpora a relevância intrínseca do pacote, reconhecendo que um pacote muito utilizado é, por si só, um alvo de maior valor. Um pacote só alcança o topo dessa lista quando reúne as três condições simultaneamente: é grave, é central o bastante para contaminar muitos outros e é, em si mesmo, amplamente adotado.

A Figura 4 apresenta os pacotes com maiores escores de risco, e a Tabela 2 detalha os oito pacotes mais críticos, decompondo o escore em seus componentes. O resultado é eloquente quanto à concentração de risco no ecossistema: trata-se majoritariamente de bibliotecas de infraestrutura de rede, criptografia, serialização e computação numérica — exatamente os blocos sobre os quais boa parte do software Python é construída.

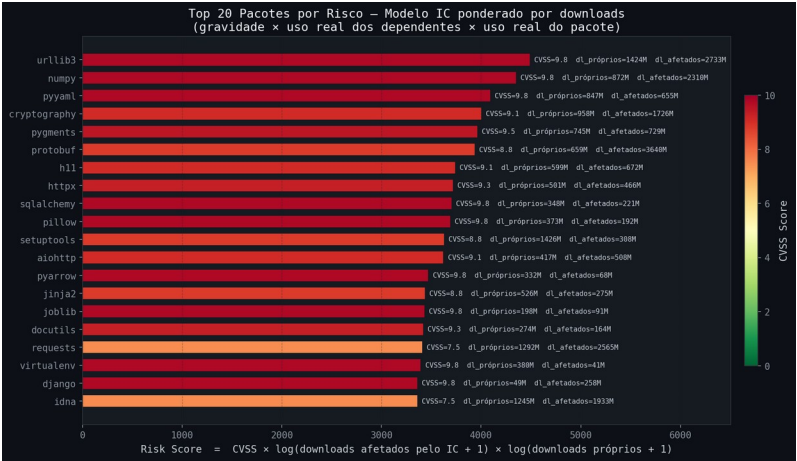


Figura 4. Pacotes com maiores escores de risco (risk_score), combinando severidade CVSS, impacto esperado da propagação e popularidade própria.

Pacote	CVSS	Nº vulns	Downloads próprios	Alcance IC (médio)	Alcance BFS	risk_score
urllib3	9,8	28	1.424.208.495	955	14.681	4488,15
numpy	9,8	16	871.842.108	3.222	10.782	4349,72
pyyaml	9,8	8	847.114.625	907	8.699	4089,77
cryptography	9,1	28	957.699.989	497	5.522	4002,55
pygments	9,5	9	744.712.237	344	7.749	3960,48
protobuf	8,8	6	659.365.339	290	3.785	3934,11
h11	9,1	1	599.426.582	182	7.914	3738,34
httpx	9,3	2	501.349.000	914	7.173	3718,59

Tabela 2 — Pacotes mais críticos ordenados pelo escore de risco (risk_score). O alcance IC é o número esperado de pacotes atingidos pela propagação estocástica; o alcance BFS é o número de descendentes alcançáveis na estrutura do grafo.

O pacote urllib3 lidera a classificação com folga, e as razões para isso sintetizam a lógica do escore. Ele acumula CVSS máximo de 9,8, no extremo superior da escala, e nada menos que 28 vulnerabilidades conhecidas; é,

ao mesmo tempo, um dos pacotes mais baixados de todo o PyPI, com mais de 1,4 bilhão de downloads; e, como cliente HTTP de baixo nível, figura como dependência transitiva de uma parcela enorme da pilha de desenvolvimento web — de bibliotecas de requisições a frameworks completos. A coincidência de gravidade máxima, histórico extenso de falhas, adoção massiva e posição estrutural privilegiada faz dele o ponto único de maior retorno potencial para um atacante e, simetricamente, o alvo prioritário de qualquer esforço de mitigação. Logo abaixo aparecem `numpy`, `pyyaml` e `cryptography`, que reproduzem o mesmo padrão em domínios distintos: computação científica, serialização de dados e primitivas criptográficas.

A comparação entre o alcance estimado pelo modelo IC e o alcance estrutural medido por busca em largura (BFS) revela um dos achados mais importantes do trabalho e está ilustrada na Figura 5. O alcance BFS conta simplesmente quantos pacotes são, em princípio, alcançáveis a partir de uma fonte seguindo as arestas de dependência; ele responde à pergunta "quem poderia, na pior das hipóteses, ser atingido?". O alcance IC, por outro lado, estima quantos pacotes seriam de fato atingidos em uma propagação típica, ponderando cada passo pela probabilidade de a falha realmente se transmitir. A diferença entre os dois é grande e sistemática: `urllib3` alcança 14.681 pacotes pela métrica estrutural de BFS, mas seu alcance esperado pelo IC é de apenas cerca de 955 — uma redução de mais de uma ordem de grandeza. A leitura é clara: alcance estrutural não é o mesmo que impacto prático. Nem toda dependência se "ativa" em uma propagação real; muitas arestas correspondem a vínculos periféricos ou pouco críticos, que raramente conduziriam a falha adiante. O IC ponderado captura precisamente essa distinção, oferecendo uma estimativa de impacto muito mais realista do que a mera contagem de descendentes — e mostrando que tomar o alcance estrutural como medida de risco superestimaria gravemente o efeito de uma falha.

O caso de `numpy` ilustra o ponto pelo ângulo oposto e merece atenção. Embora seu alcance estrutural por BFS (10.782) seja menor que o de `urllib3`, seu alcance esperado pelo IC é o maior de toda a tabela, em torno de 3.222 pacotes. Em outras palavras, `numpy` tende a propagar uma falha de forma muito mais efetiva do que `urllib3`, apesar de alcançar estruturalmente menos pacotes. A explicação está na natureza de suas dependências: `numpy` é o alicerce da pilha científica de Python, e os pacotes que dele dependem em geral o fazem de maneira essencial e não substituível — `pandas`, `scikit-learn`, `scipy` e tantos outros não funcionam sem ele. Suas arestas de dependência têm, portanto, probabilidade de propagação alta, e o modelo IC reflete isso. Esse contraste entre os dois pacotes é a melhor evidência de que a posição no grafo importa não só em quantidade, mas em qualidade: a centralidade de `numpy` na pilha científica o torna um propagador mais perigoso de fato, ainda que estatísticas estruturais brutas sugiram o contrário.

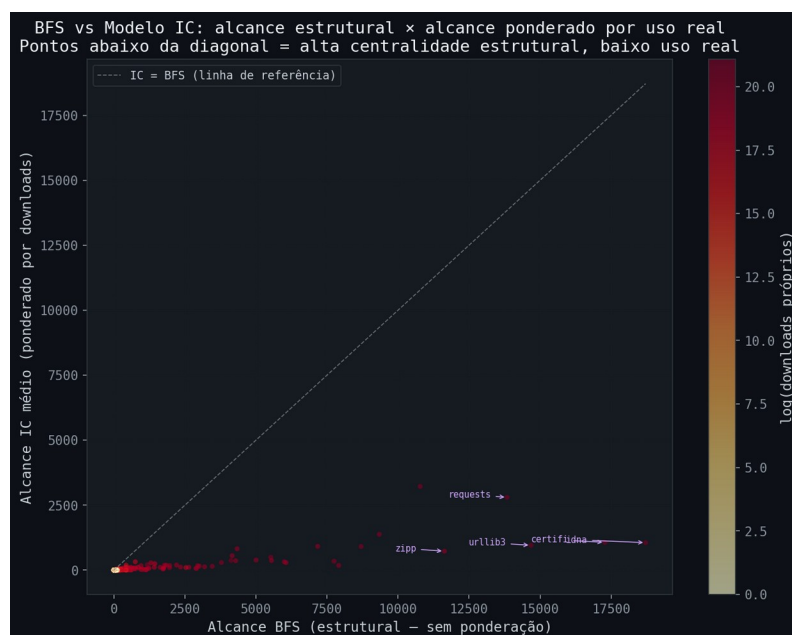


Figura 5. Alcance esperado pela Cascata Independente (IC) versus alcance estrutural por busca em largura (BFS). O alcance BFS superestima sistematicamente o impacto efetivo de uma propagação.

A Figura 6 dispõe os pacotes vulneráveis no plano definido por número de downloads e gravidade CVSS, e torna visual um princípio que o escore de risco já formaliza: gravidade alta, isolada, não basta para caracterizar risco sistêmico. Há pacotes com CVSS no topo da escala que, por serem pouco utilizados, permanecem em segundo plano — uma falha grave em um pacote obscuro tem efeito limitado simplesmente porque há pouca superfície exposta. Inversamente, pacotes de enorme adoção mas com vulnerabilidades de severidade moderada também não dominam o ranking isoladamente. O risco real emerge do cruzamento das duas dimensões: é no quadrante de alta gravidade e alta adoção que se concentram os pacotes verdadeiramente críticos, e é exatamente esse cruzamento que

o produto $CVSS \cdot \text{popularidade}$ no escore de risco busca capturar. A dispersão deixa claro que qualquer priorização baseada em um só eixo — apenas severidade, como fazem muitos painéis de segurança, ou apenas popularidade — levaria a decisões equivocadas sobre onde investir esforço de auditoria.

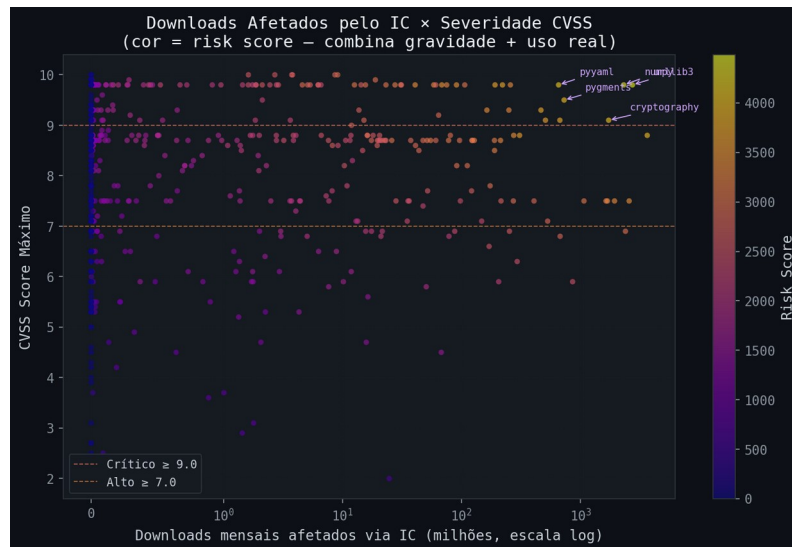


Figura 6. Dispersão dos pacotes vulneráveis por número de downloads e gravidade CVSS. O risco sistêmico concentra-se no cruzamento entre alta severidade e ampla adoção, não em qualquer eixo isolado.

Para tornar concreto o mecanismo de propagação que o modelo quantifica em agregado, a Figura 7 exibe um exemplo de cascata a partir de um pacote-hub. A figura traça como uma falha originada em um único pacote central se ramifica pelos seus dependentes diretos e, destes, para os dependentes de segunda e terceira ordem, com a intensidade de cada vínculo refletindo a probabilidade de propagação atribuída pelo modelo. O que a ilustração evidencia é o efeito de amplificação característico dos hubs: o comprometimento de um nó situado próximo à raiz da pilha de dependências não permanece local, mas se espalha em leque, atingindo dezenas a centenas de pacotes que jamais referenciaram diretamente a fonte da falha. É esse efeito de alavancagem topológica que transforma uma vulnerabilidade pontual em um problema de cadeia de suprimentos.

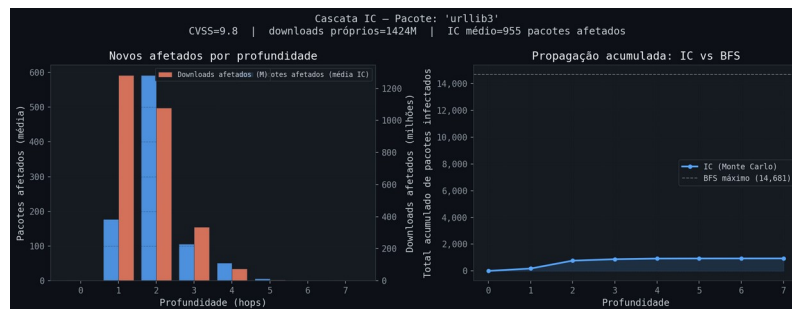


Figura 7. Exemplo de cascata de propagação de uma falha a partir de um pacote-hub. A falha se ramifica em níveis sucessivos de dependentes, evidenciando o efeito de amplificação dos nós centrais.

Reunidos, esses resultados respondem à motivação que abriu o projeto. A segurança de um ecossistema de software não é a soma das seguranças de seus pacotes tomados isoladamente; ela é determinada, em grande medida, por um pequeno conjunto de pacotes que concentram simultaneamente gravidade, centralidade e adoção. Os pacotes no topo da Tabela 2 são justamente os pontos em que o ecossistema é mais frágil e, por isso mesmo, aqueles em que qualquer esforço de mitigação — manter versões atualizadas, conduzir auditorias de código, financiar a manutenção dos projetos — oferece o maior retorno marginal. Proteger `urllib3`, `numpy` ou `cryptography` tem efeito sistêmico desproporcional à sua contagem unitária, porque sua segurança se propaga, pelo mesmo grafo que propagaria suas falhas, para tudo o que deles depende. A combinação de modelagem topológica e dados de vulnerabilidade desenvolvida neste trabalho oferece, assim, um critério objetivo e quantitativo para direcionar recursos finitos de segurança aos poucos pontos onde eles mais importam.

7. Detecção de comunidades

A organização interna do ecossistema foi investigada por meio de detecção de comunidades, aplicada ao maior componente fracamente conexo do grafo, tratado como não-direcionado. Empregou-se o algoritmo de Louvain, que maximiza a modularidade de forma gulosa e hierárquica, oferecendo bom equilíbrio entre qualidade da partição e escalabilidade para grafos desta dimensão. A partição resultante reúne os pacotes em 63 comunidades e atinge modularidade $Q = 0,50$. Como valores de Q acima de 0,3 já são tomados como indício de estrutura modular significativa, o resultado revela um ecossistema fortemente compartimentado em blocos internamente coesos e externamente esparsos. Os tamanhos das comunidades são, contudo, muito heterogêneos: a maior reúne 7.749 nós e a menor apenas 3, com mediana de 16 e média de 637. Esse contraste entre média e mediana evidencia uma distribuição de cauda longa, em que poucas comunidades volumosas convivem com uma vasta população de agrupamentos pequenos e especializados.

Para conectar essa estrutura ao objetivo de segurança do projeto, cada comunidade foi cruzada com os pacotes vulneráveis identificados na Seção 6. A Tabela 3 sintetiza as cinco comunidades de maior escore de risco, reportando o número de pacotes, a quantidade e a fração de pacotes vulneráveis, o CVSS máximo observado e o tema funcional dominante de cada bloco.

Comunidade	Nº pacotes	Nº vulneráveis	Fração vulnerável	CVSS máx	Tema dominante
28	32	3	9,4%	8,8	ERP Tryton (simpleeval, trytond)
9	415	26	6,3%	9,9	Zope/Plone + setuptools
11	481	20	4,2%	9,8	jsonschema / Ansible / OpenStack
0	5062	167	3,3%	9,8	núcleo web/base (urllib3, flask, requests, six)
32	1090	34	3,1%	9,9	stack de dados/Airflow (openpyxl, paramiko, pymysql)

Tabela 3 — Comunidades de maior escore de risco, com volume de pacotes, incidência de vulnerabilidades e tema funcional dominante.

A leitura da Tabela 3 exige atenção ao critério de ordenação. O escore de risco por comunidade pondera o resultado pela fração de pacotes vulneráveis, de modo que a métrica mede a concentração de risco e não o seu volume. Por essa razão, o topo do ranking é ocupado por comunidades pequenas e densamente comprometidas: a comunidade 28, núcleo do ERP Tryton, lidera com 9,4% de pacotes vulneráveis ainda que reúna apenas 32 pacotes, e é seguida pelos blocos Zope/Plone (6,3%) e jsonschema/Ansible/OpenStack (4,2%). São agrupamentos especializados em que uma única falha grave eleva substancialmente a proporção comprometida do bloco. A Figura 8 ilustra a distribuição de tamanhos das comunidades e a Figura 9, o escore de risco por comunidade.

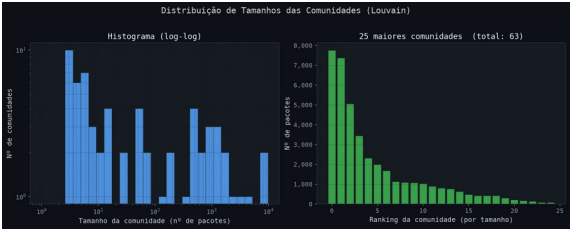


Figura 8. Distribuição do tamanho das 63 comunidades detectadas pelo algoritmo de Louvain, em escala logarítmica. Poucas comunidades volumosas coexistem com uma longa cauda de agrupamentos pequenos.

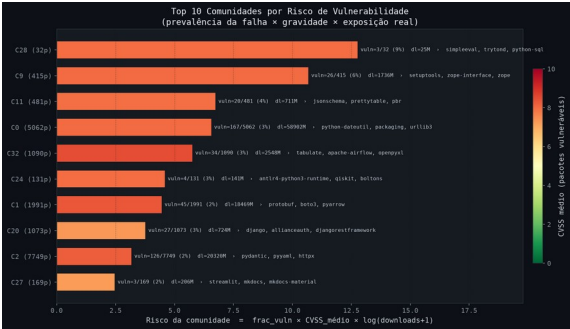


Figura 9. Score de risco por comunidade. O ranking, ponderado pela fração de pacotes vulneráveis, destaca comunidades pequenas e densamente comprometidas, como a comunidade 28 (ERP Tryton).

Esse retrato se inverte quando a perspectiva passa da fração para o volume absoluto de pacotes vulneráveis. Sob esse ângulo, a comunidade 0 — o núcleo web/base que congrega urllib3, flask, requests e six — concentra sozinha 167 pacotes vulneráveis, e a comunidade 32, correspondente ao stack de dados em torno do Airflow (openpyxl, paramiko, pymysql), soma outros 34. São justamente essas as comunidades de maior impacto sistêmico, pois abrigam os hubs de altíssimo grau de entrada identificados na Seção 6, cuja falha se propaga a milhares de dependentes. As duas leituras são, portanto, complementares: a fração aponta onde o risco está mais concentrado,

enquanto o volume aponta onde uma vulnerabilidade alcança o maior número de pacotes. A Figura 10 traz uma visão da rede de comunidades, evidenciando os blocos funcionais e suas interconexões.

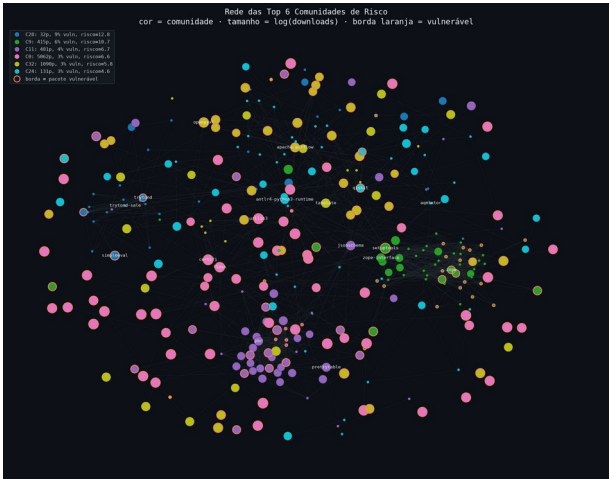


Figura 10. Visão da rede de comunidades do maior componente do grafo. Cada cor representa um módulo funcional coeso, com arestas internas densas e ligações esparsas entre blocos.

Em conjunto, esses achados conferem significado prático à modularidade elevada ($Q = 0,50$): longe de ser apenas uma propriedade abstrata, ela reflete a organização do ecossistema em módulos funcionais bem delimitados — o núcleo web, o ramo científico e de dados, os sistemas de ERP e as ferramentas de automação. Essa compartimentação tem valor operacional direto, pois permite localizar e priorizar a mitigação por bloco, concentrando esforços ora nas comunidades de maior concentração de risco, ora nas de maior impacto sistêmico, conforme o objetivo de segurança a ser atendido.

8. Validação do modelo

A validade de um ranking de risco não se demonstra por construção, mas por confronto com evidência externa ao próprio modelo. Adotou-se, por isso, uma estratégia de validação tripla, deliberadamente apoiada em fontes independentes entre si. A primeira frente mede a capacidade de recuperação do ranking por meio de Precision@K e Recall@K , comparando a ordenação produzida pelo escore de risco (risk_score) contra um gabarito curado de pacotes associados a incidentes de segurança reais. A segunda frente avalia a convergência do escore com proxies de risco obtidos por caminhos independentes — o número de downloads, como indicador de exposição, e o CVSS, como indicador de gravidade — por meio da correlação de postos de Spearman. A terceira frente examina, em estudos de caso, se o alcance de propagação estimado sobre o grafo (busca em largura, BFS, e Cascata Independente, IC) é coerente com o papel estrutural de pacotes notoriamente críticos. O gabarito reúne treze pacotes com vulnerabilidades documentadas — `urllib3`, `requests`, `pyyaml`, `cryptography`, `jinja2`, `flask`, `django`, `setuptools`, `numpy`, `pillow`, `paramiko`, `lxml` e `werkzeug` — e a totalidade dos treze foi localizada dentro das cinquenta primeiras posições do ranking. Registre-se, como ressalva metodológica obrigatória, que a associação entre cada pacote e seus CVEs foi curada manualmente; antes da versão definitiva do relatório essa lista deve ser conferida nas bases OSV e NVD, de modo a eliminar imprecisões de identificador ou de versão afetada.

A Tabela 4 sintetiza o desempenho de recuperação nos cortes $K = 10, 20$ e 50 . A leitura conjunta de precisão e revocação é mais informativa do que qualquer métrica isolada, sobretudo em um cenário no qual o conjunto de referência é pequeno em relação ao espaço de busca.

K	Precision@K	Recall@K	Acertos
10	0,50	0,38	5
20	0,45	0,69	9
50	0,26	1,00	13

Tabela 4 — Precision@K e Recall@K do ranking por risk_score contra o gabarito de treze pacotes com incidentes reais; a coluna Acertos indica quantos desses pacotes aparecem entre as K primeiras posições.

O resultado decisivo está no corte $K = 50$: a revocação atinge 1,00, ou seja, o modelo capturou a totalidade dos treze pacotes de incidente conhecido dentro do top-50, sem deixar nenhum de fora. A precisão correspondente, de

0,26, não deve ser lida como fragilidade do modelo, mas como artefato direto do tamanho do gabarito. Com apenas treze pacotes no conjunto de referência, a precisão máxima teoricamente atingível em $K = 50$ é $13/50 = 0,26$; o modelo, portanto, alcançou exatamente o teto imposto pela aritmética da própria métrica, e nenhum ranking — por melhor que fosse — poderia superá-lo nesse corte. Nos cortes menores a precisão sobe (0,50 em $K = 10$), confirmando que os acertos se concentram no topo, mas é a revocação que carrega a informação relevante neste experimento: o modelo não apenas ordena bem, como não perde pacotes efetivamente perigosos.

A segunda frente de validação examina se o escore de risco é apenas um disfarce para um único fator já conhecido. As correlações de Spearman foram calculadas sobre os cinquenta pacotes ranqueados — um conjunto já pré-selecionado por risco, o que introduz restrição de amplitude e tende a atenuar os coeficientes observados. Ainda assim, o `risk_score` correlaciona-se moderadamente com o número de downloads ($\rho = +0,55$; $p = 3,1 \times 10^{-5}$), de forma mais fraca com o CVSS ($\rho = +0,28$; $p = 0,052$) e de modo apreciável com o alcance estimado pela Cascata Independente ($\rho = +0,59$). O ponto central é que nenhuma dessas correlações se aproxima de 1,0: o ranking não é trivialmente explicado por popularidade, por gravidade nem por propagação tomadas isoladamente. Os três componentes contribuem de maneira independente para a ordenação final, o que é precisamente o comportamento desejado de um escore que pretende combinar gravidade, exposição e alcance estrutural em uma única medida. A Figura 11 detalha o comportamento de `Precision@K` e `Recall@K` em função de K , e a Figura 12 dispõe a matriz de correlações entre o escore e os proxies independentes.

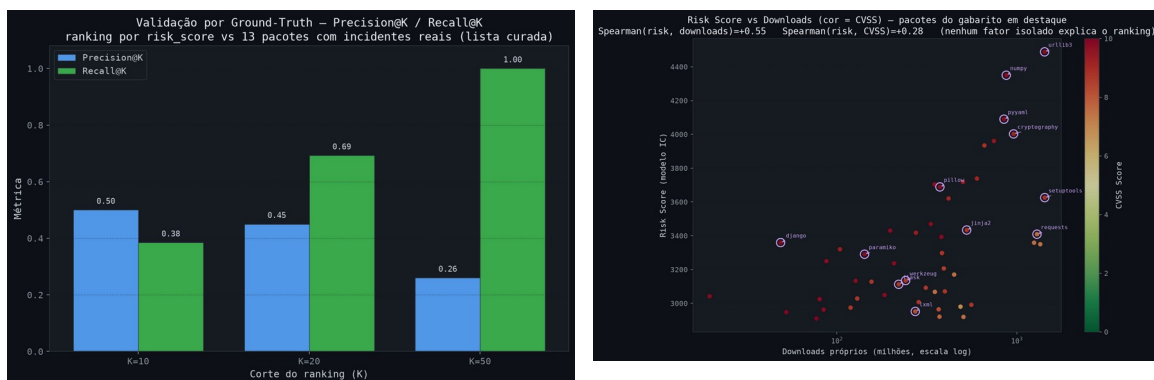


Figura 11. Precision@K e Recall@K do ranking em função do corte K: a revocação satura em 1,00 em $K = 50$, enquanto a precisão decresce por efeito do tamanho do gabarito.

Figura 12. Matriz de correlações de Spearman entre o risk-score e os proxies independentes (downloads, CVSS e alcance IC), evidenciando contribuição não redundante dos três fatores.

A terceira frente ancora a validação em casos concretos, cruzando o incidente real com o alcance de propagação medido sobre o grafo. O pacote `urllib3`, primeiro colocado no ranking, concentra a CVE-2023-43804 — vazamento do cabeçalho Cookie em redirecionamentos — e exibe alcance BFS de 14.681 dependentes, profundidade máxima 19 e alcance esperado pela Cascata Independente de aproximadamente 955, coerente com seu papel de dependência transitiva quase universal. O pacote `pyyaml`, em terceiro lugar, responde pela CVE-2020-14343 — execução remota de código via `yaml.load()` inseguro — com alcance BFS de 8.699. Já `requests`, em décimo sétimo lugar, carrega a CVE-2023-32681 — vazamento do cabeçalho Proxy-Authorization — e apresenta alcance BFS de 13.839 e alcance IC de aproximadamente 2.801, este último superior ao de `urllib3` apesar de um CVSS menor, o que ilustra como o componente de propagação pondera o impacto esperado pelo uso efetivo dos dependentes. Em todos os três casos o alcance BFS reproduz exatamente os valores registrados em `vuln_stats.json`, confirmando a consistência interna entre o módulo de validação e a análise de propagação descrita na Seção 6.

Em conjunto, as três frentes convergem para a mesma conclusão: o modelo ordena corretamente os pacotes de risco real, recuperando-os por completo no top-50 e combinando gravidade, popularidade e propagação de forma não redundante — com a ressalva de que o gabarito é pequeno, com apenas treze pacotes, limitação que será retomada na Conclusão.

9. Conclusão

Este trabalho modelou o ecossistema do PyPI como um grafo direcionado de 54.406 pacotes e 178.193 dependências e, ao cruzar a topologia da rede com dados reais de download e com vulnerabilidades conhecidas, identificou os pacotes e as comunidades de maior risco sistêmico. A síntese a seguir relaciona cada objetivo proposto à seção em que foi atendido e ao resultado obtido.

Tabela 5 — Cumprimento dos objetivos.

Objetivo	Onde foi atendido	Resultado
Construir um grafo de dependências do PyPI	Seção 3 + Algoritmo 1	Grafo com 54.406 nós e 178.193 arestas
Identificar os pacotes de maior centralidade (pilares)	Seção 5	Hubs: numpy (in-degree 6.479), requests, pydantic, pandas, pyyaml
Mapear vulnerabilidades conhecidas (OSV/GitHub Advisory)	Seção 6	648 pacotes vulneráveis (1,2%), com CVSS computado via biblioteca cvss
Simular falhas e medir o efeito cascata	Seções 4 e 6	Independent Cascade + risk_score; topo crítico: urllib3, numpy, pyyaml
Visualizar as comunidades mais afetadas	Seção 7	Louvain: 63 comunidades (Q = 0,50); núcleo web/base e o stack de dados mais expostos

Cabe registrar que a coleta de dados apoiou-se na API JSON do PyPI, e não no Libraries.io ou no BigQuery previstos na proposta inicial, sem prejuízo ao cumprimento dos objetivos.

Os resultados devem ser lidos à luz de algumas limitações. O grafo analisado constitui um subconjunto do PyPI, de modo que os alcances de propagação reportados representam um limite inferior do efeito real; o modelo de Independent Cascade depende da parametrização adotada — probabilidades de contágio ponderadas pelo volume de downloads e 500 simulações por cenário —, sendo, portanto, sensível a essas escolhas; e o gabarito de validação é pequeno (13 pacotes) e curado manualmente, ainda a confirmar de forma sistemática em bases como OSV e NVD.

Como desdobramentos, propõe-se expandir a cobertura do grafo para a totalidade do índice, calibrar empiricamente as probabilidades do modelo de cascata e ampliar o gabarito de validação, conferindo maior robustez estatística às conclusões.

10. Referências

1. TARJAN, R. E. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, v. 1, n. 2, p. 146–160, 1972.
2. BLONDEL, V. D.; GUILLAUME, J.-L.; LAMBIOTTE, R.; LEFEBVRE, E. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*, P10008, 2008.
3. KEMPE, D.; KLEINBERG, J.; TARDOS, É. Maximizing the spread of influence through a social network. In: *KDD '03*, p. 137–146, 2003.
4. HAGBERG, A.; SCHULT, D.; SWART, P. Exploring network structure, dynamics, and function using NetworkX. In: *Proceedings of the 7th Python in Science Conference (SciPy)*, 2008.
5. OSV — Open Source Vulnerabilities Database. Disponível em: <https://osv.dev>. Acesso em: 22 jun. 2026.
6. GITHUB ADVISORY DATABASE. Disponível em: <https://github.com/advisories>. Acesso em: 22 jun. 2026.
7. PYTHON PACKAGE INDEX (PyPI). Disponível em: <https://pypi.org>. Acesso em: 22 jun. 2026.
8. FIRST. Common Vulnerability Scoring System (CVSS) v3.1 Specification. Disponível em: <https://www.first.org/cvss>. Acesso em: 22 jun. 2026.
9. PYTHON SOFTWARE FOUNDATION. PEP 503 — Normalized package names; PEP 508 — Dependency specification. Disponível em: <https://peps.python.org>. Acesso em: 22 jun. 2026.
10. HUGOVK. top-pypi-packages. Disponível em: <https://hugovk.github.io/top-pypi-packages>. Acesso em: 22 jun. 2026.